

Chip in, big win

UVM Driver 常考题 / 真题汇总

Part 2 / 3 · 题目 13 ~ 24

SystemVerilog · UVM 1.2 / IEEE 1800.2

本册涵盖题目 13-24：时序与 Reset 续（题13-15）、典型协议 Driver（题16-22）、Pipeline 与多笔在途（题23-24）。

题目 13 用 reset_phase 组织 Driver 线程

难度：高级

标签：UVM phases、线程管理、reset 流程、disable fork、重入控制

【题目描述】

使用 UVM 多 phase 结构组织 driver 的生命周期，使 reset 处理与 UVM 调度框架深度融合。

各 phase 职责分配：

1. pre_reset_phase：杀掉上一轮 main_phase 遗留的 drive 线程（用 process 对象追踪并 kill），防止重入时多份线程并发；

2. reset_phase：驱动接口到 idle/默认值，等待 reset_n 释放，raise/drop objection 控制 phase 时长；

3. main_phase：正常 pull/drive 循环，fork process_trans + monitor_reset（join_any），reset 发生时 monitor_reset 返回触发 main_phase 结束；

4. 禁止重入时多份并发 drive 线程：通过 process 句柄追踪上一轮线程并在 pre_reset_phase 中 kill。

【完整答案代码】

```
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    process drive_proc; // 追踪 main_phase drive 线程，防止重入
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "virtual interface not found")
    endfunction

    // pre_reset_phase：杀掉上一轮遗留线程
    task pre_reset_phase(uvm_phase phase);
        if (drive_proc != null) begin
            drive_proc.kill();
            drive_proc = null;
            `uvm_info("DRV", "pre_reset: killed previous drive thread", UVM_LOW)
        end
    endtask

    // reset_phase：驱动 idle、等待 reset 释放
    task reset_phase(uvm_phase phase);
        phase.raise_objection(this);
        v_if.drive_cb.data <= 8'd0;
```

```
        `uvm_info("DRV", "reset_phase: driving idle", UVM_LOW)
        @(posedge v_if.mon_cb.reset);
        @(v_if.drive_mp.drive_cb);
        `uvm_info("DRV", "reset_phase: reset released", UVM_LOW)
        phase.drop_objection(this);
    endtask

    // main_phase：正常 pull/drive，套用模版 fork 结构
    task main_phase(uvm_phase phase);
        phase.raise_objection(this);
        fork
            begin
                drive_proc = process::self();
                process_trans();
            end
            monitor_reset_for_phase();
        join_any
        disable fork;
        drive_proc = null;
        phase.drop_objection(this);
    endtask

    task monitor_reset_for_phase();
        @(negedge v_if.mon_cb.reset);
        `uvm_info("DRV", "Reset in main_phase, ending phase", UVM_LOW)
    endtask

    task process_trans();
        my_transaction req, rsp;
        forever begin
            seq_item_port.get_next_item(req);
            drive_item(req);
            void'($cast(rsp, req.clone()));
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.item_done(rsp);
        end
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
    endtask

    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
```

```
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
    endtask
endclass
```

【模版套用说明】

pre_reset_phase 负责杀掉上一轮线程防止重入。main_phase 用 fork/join_any + disable fork 结构（来自模版）。reset_phase raise/drop objection 保证 phase 时序正确。process_trans 不变，完全复用模版。

题目 14 accept_tr/begin_tr/end_tr 事务记录

难度：高级

标签：accept_tr、begin_tr、end_tr、end_event、pipeline

【题目描述】

实现三段式事务记录，精确标记事务在 driver 内的生命周期三个时间点：

1. accept_tr(req, $realtime)：事务被 driver 接受（get() 返回后立即），表示 driver 已"拥有"该事务；

2. begin_tr(req)：事务真正开始执行（等到 grant 到来/地址相位启动时），表示总线上有实际活动；

3. end_tr(req, $realtime)：事务完全完成（最后一个数据拍完成或响应收到），触发 end_event；

sequence 可通过 req.end_event.wait_on() 等待真正完成，实现精细的 pipeline 流控。

注意：begin_tr 返回值是事务句柄，用 void'() 避免未使用警告；end_tr 触发 end_event 是自动行为，不需要手动 trigger。

【完整答案代码】

```
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "virtual interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        fork
            process_trans();
            monitor_reset();
        join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.data <= 8'd0;
        end
    endtask

    task process_trans();
```

```
        my_transaction req, rsp;
        forever begin
            // 使用 get()：pipeline 语义，提前释放 sequence
            seq_item_port.get(req);

            // 时间点1：accept（已取到 item）
            accept_tr(req, $realtime);

            // 等待 grant（模拟两阶段协议中的仲裁等待）
            @(v_if.drive_mp.drive_cb);

            // 时间点2：begin（真正开始执行，grant 后）
            void'(begin_tr(req, "my_driver"));

            // 执行事务
            drive_item(req);

            // 时间点3：end（完全完成，触发 end_event）
            end_tr(req, $realtime);

            // 回传响应
            $cast(rsp, req.clone());
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.put_response(rsp);
        end
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
    endtask

    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
    endtask
endclass
```

【模版套用说明】

accept_tr/begin_tr/end_tr 三个调用点分别对应 item 取到、执行开始、执行结束。end_tr 自动触发 req.end_event，sequence 可 wait_on() 等待。使用 get() 实现 pipeline，sequence 可在 accept 后提前解阻塞，然后选择 wait end_event 等真正完成。

题目 15 Ready 超时检测与错误注入

难度：中级

标签：timeout、错误注入、健壮性、reset 豁免、日志

【题目描述】

实现带超时检测的 driver，增强对 DUT 无响应场景的鲁棒性。

设计要点：

1. 超时逻辑封装在独立 task wait_ready_with_timeout()，返回 bit 表示成功/超时，与正常驱动逻辑解耦；

2. timeout_cycles=0 表示不启用超时（永远等待），便于功能验证阶段使用；

3. Reset 豁免：等待 ready 的循环内检测 reset_n==0，立即返回成功（避免 reset 期间误报超时）；

4. 超时时：uvm_error 打印事务关键信息（addr/type），拉回总线 idle；

5. 无论正常/超时/reset，都在 process_trans 末尾统一调用 item_done，保证 item 生命周期配对。

【完整答案代码】

```
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    int unsigned timeout_cycles = 200; // 0 = 不启用
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "virtual interface not found")
        void'(uvm_config_db#(int unsigned)::get(
            this, "", "timeout_cycles", timeout_cycles));
    endfunction

    task run_phase(uvm_phase phase);
        fork process_trans(); monitor_reset(); join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.data <= 8'd0;
        end
    endtask

```

```
    task process_trans();
        my_transaction req, rsp;
        bit ok;
        forever begin
            seq_item_port.get_next_item(req);

            drive_item_with_timeout(req, ok);

            if (!ok) begin
                `uvm_error("DRV_TIMEOUT",
                    $sformatf("Timeout waiting ready! addr=0x%0h type=%0d @%0t",
                              req.addr, req.type, $time))
                v_if.drive_cb.data <= 8'd0;
            end

            // 无论成功/超时：统一 item_done
            void'($cast(rsp, req.clone()));
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.item_done(rsp);
        end
    endtask

    // 带超时的驱动 task
    task drive_item_with_timeout(
        input my_transaction trans,
        output bit ok
    );
        ok = 1;
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
        begin
            int cnt = 0;
            while (!v_if.drive_cb.ready) begin
                // Reset 豁免：reset 期间直接返回
                if (!v_if.mon_cb.reset) return;
                @(v_if.drive_mp.drive_cb);
                if (timeout_cycles > 0 && ++cnt >= timeout_cycles) begin
                    ok = 0; return;
                end
            end
        end
    endtask

    task drive_item(my_transaction trans);
        bit ok;
        drive_item_with_timeout(trans, ok);
    endtask
```

```
    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        bit ok;
        drive_item_with_timeout(trans, ok);
    endtask
endclass
```

【模版套用说明】

超时逻辑封装在 drive_item_with_timeout 中，output bit ok 返回结果。reset 豁免通过检测 mon_cb.reset 实现。item_done 在 process_trans 末尾无条件调用。timeout_cycles=0 时 while 条件永不触发超时分支。

题目 16 APB Master Driver（含 wait states）

难度：中级

标签：APB、两阶段、PREADY wait、响应回填、时序同步

【题目描述】

实现符合 AMBA APB 协议规范的 master driver，精确实现两阶段握手和 wait states。

APB 协议时序要点：

1. Setup phase（T1 拍）：驱动 PSEL=1, PENABLE=0，建立 PADDR/PWRITE/PWDATA/PSTRB，稳定一个时钟；

2. Access phase（T2 拍起）：驱动 PENABLE=1，保持所有地址控制信号不变；

3. Wait states：PREADY=0 时持续保持 PENABLE=1 和所有信号，不得更新，do-while 等待 PREADY；

4. Transfer 完成（PREADY=1）：读事务采样 PRDATA 和 PSLVERR，回传 rsp；

5. 完成后 PSEL=0, PENABLE=0，总线回 idle，支持 back-to-back（下一笔直接进入 setup）；

6. reset 时立即拉所有信号回 idle（通过 monitor_reset 线程）。

【完整答案代码】

```
class my_driver extends uvm_driver#(apb_transaction);
    virtual apb_interface v_if;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual apb_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "APB interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        fork
            process_trans();
            monitor_reset();
        join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.PSEL <= 1'b0;
            v_if.drive_cb.PENABLE <= 1'b0;
            v_if.drive_cb.PADDR <= '0;
            v_if.drive_cb.PWDATA <= '0;
```

```
            v_if.drive_cb.PWRITE <= 1'b0;
        end
    endtask

    task process_trans();
        apb_transaction req, rsp;
        forever begin
            seq_item_port.get_next_item(req);
            drive_item(req);
            void'($cast(rsp, req.clone()));
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.item_done(rsp);
        end
    endtask

    task drive_item(apb_transaction trans);
        if (trans.pwrite) write_item(trans);
        else read_item(trans);
    endtask

    // APB 写：setup phase -> access phase -> wait PREADY
    task write_item(apb_transaction trans);
        // Setup phase（第1拍）
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.PSEL <= 1'b1;
        v_if.drive_cb.PENABLE <= 1'b0;
        v_if.drive_cb.PWRITE <= 1'b1;
        v_if.drive_cb.PADDR <= trans.paddr;
        v_if.drive_cb.PWDATA <= trans.pwdata;
        v_if.drive_cb.PSTRB <= trans.pstrb;
        // Access phase（第2拍起）
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.PENABLE <= 1'b1;
        // Wait states：保持所有信号，等 PREADY
        do @(v_if.drive_mp.drive_cb);
        while (!v_if.drive_cb.PREADY);
        v_if.drive_cb.PSEL <= 1'b0;
        v_if.drive_cb.PENABLE <= 1'b0;
    endtask

    // APB 读：setup -> access -> wait PREADY -> 采样 PRDATA
    task read_item(apb_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.PSEL <= 1'b1;
        v_if.drive_cb.PENABLE <= 1'b0;
        v_if.drive_cb.PWRITE <= 1'b0;
        v_if.drive_cb.PADDR <= trans.paddr;
```

```
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.PENABLE <= 1'b1;
        do @(v_if.drive_mp.drive_cb);
        while (!v_if.drive_cb.PREADY);
        trans.prdata = v_if.drive_cb.PRDATA;
        trans.pslverr = v_if.drive_cb.PSLVERR;
        v_if.drive_cb.PSEL <= 1'b0;
        v_if.drive_cb.PENABLE <= 1'b0;
    endtask
endclass
```

【模版套用说明】

APB 两阶段精确映射到 write_item/read_item：第1个 @clk 为 setup，第2个 @clk 起为 access，do-while 等 PREADY 实现任意 wait states。模版 process_trans + monitor_reset 骨架不变，仅接口类型改为 apb_interface。

题目 17 AXI-Lite 写通道 Driver（AW/W/B）

难度：高级

标签：AXI-Lite、写通道、backpressure、并发线程、响应

【题目描述】

实现 AXI-Lite master 写通道 driver，并行驱动 AW/W 两个独立通道。

AXI-Lite 写协议要点：

1. AW（写地址）通道和 W（写数据）通道可以同时发起，互相独立握手（fork join）；

2. 每个通道独立等待各自的 READY 信号，期间保持 VALID 和数据稳定（backpressure 处理）；

3. B（写响应）通道：master 驱动 BREADY=1 表示准备接收响应，等待 BVALID 后采样 BRESP；

4. 同一事务的 AW/W fork 执行后 join，保证两个通道都完成后才进入 B 通道等待；

5. monitor_reset 拉低所有 VALID 和 READY 信号防止残留。

【完整答案代码】

```
class my_driver extends uvm_driver#(axi_lite_transaction);
    virtual axi_lite_interface v_if;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual axi_lite_interface)::get(
                this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "AXI-Lite interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        fork
            process_trans();
            monitor_reset();
        join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.AWVALID <= 1'b0;
            v_if.drive_cb.WVALID <= 1'b0;
            v_if.drive_cb.BREADY <= 1'b0;
        end
    endtask

    task process_trans();
```

```
        axi_lite_transaction req, rsp;
        forever begin
            seq_item_port.get_next_item(req);
            drive_item(req);
            void'($cast(rsp, req.clone()));
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.item_done(rsp);
        end
    endtask

    // drive_item：AW/W 并行，然后等 B 响应
    task drive_item(axi_lite_transaction trans);
        fork
            send_aw(trans);
            send_w(trans);
        join
        recv_b(trans);
    endtask

    task send_aw(axi_lite_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.AWVALID <= 1'b1;
        v_if.drive_cb.AWADDR <= trans.addr;
        do @(v_if.drive_mp.drive_cb);
        while (!v_if.drive_cb.AWREADY);
        v_if.drive_cb.AWVALID <= 1'b0;
    endtask

    task send_w(axi_lite_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.WVALID <= 1'b1;
        v_if.drive_cb.WDATA <= trans.data;
        v_if.drive_cb.WSTRB <= trans.wstrb;
        do @(v_if.drive_mp.drive_cb);
        while (!v_if.drive_cb.WREADY);
        v_if.drive_cb.WVALID <= 1'b0;
    endtask

    task recv_b(axi_lite_transaction trans);
        v_if.drive_cb.BREADY <= 1'b1;
        do @(v_if.drive_mp.drive_cb);
        while (!v_if.drive_cb.BVALID);
        trans.bresp = v_if.drive_cb.BRESP;
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.BREADY <= 1'b0;
    endtask

```

```
    task read_item(axi_lite_transaction trans); endtask
    task write_item(axi_lite_transaction trans); endtask
endclass
```

【模版套用说明】

drive_item 内 fork AW/W 并行握手，join 后再 recv_b。模版的 process_trans + monitor_reset 骨架不变；drive_item 拆成三个子 task 对应三个 AXI-Lite 写通道，结构清晰可测。

题目 18 AXI-Lite 读通道 Driver（AR/R）

难度：高级

标签：AXI-Lite、读通道、R 通道回传、超时、队列关联

【题目描述】

实现 AXI-Lite master 读通道 driver，支持 backpressure 和超时检测。

AXI-Lite 读协议流程：

1. AR（读地址）通道：驱动 ARVALID=1 和 ARADDR，等待 ARREADY 握手，握手完成后 ARVALID=0；

2. R（读数据）通道：拉高 RREADY 表示 master 可接收，等待 RVALID，采样 RDATA/RRESP；

3. backpressure：ARREADY 可随机延迟，需保持 ARVALID 和地址稳定直到握手完成；

4. 超时检测：RVALID 在 timeout_cycles 内未到来时触发 uvm_error，防止 DUT 无响应导致仿真挂死；

5. 超时豁免：reset 期间不计超时（检测 reset_n==0 直接退出等待）。

【完整答案代码】

```
class my_driver extends uvm_driver#(axi_lite_transaction);
    virtual axi_lite_interface v_if;
    int unsigned timeout_cycles = 500;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual axi_lite_interface)::get(
                this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "AXI-Lite interface not found")
        void'(uvm_config_db#(int unsigned)::get(
            this, "", "timeout_cycles", timeout_cycles));
    endfunction

    task run_phase(uvm_phase phase);
        fork process_trans(); monitor_reset(); join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.ARVALID <= 1'b0;
            v_if.drive_cb.RREADY <= 1'b0;
        end
    endtask
```

```
    task process_trans();
        axi_lite_transaction req, rsp;
        forever begin
            seq_item_port.get_next_item(req);
            drive_item(req);
            void'($cast(rsp, req.clone()));
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.item_done(rsp);
        end
    endtask

    task drive_item(axi_lite_transaction trans);
        read_item(trans);
    endtask

    // AR 通道：驱动读地址，等待 ARREADY
    task read_item(axi_lite_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.ARVALID <= 1'b1;
        v_if.drive_cb.ARADDR <= trans.addr;
        do @(v_if.drive_mp.drive_cb);
        while (!v_if.drive_cb.ARREADY);
        v_if.drive_cb.ARVALID <= 1'b0;

        // R 通道：等 RVALID（含超时检测）
        begin
            int cnt = 0;
            v_if.drive_cb.RREADY <= 1'b1;
            while (!v_if.drive_cb.RVALID) begin
                @(v_if.drive_mp.drive_cb);
                if (!v_if.mon_cb.reset) begin
                    v_if.drive_cb.RREADY <= 1'b0;
                    return;
                end
                if (timeout_cycles > 0 && ++cnt >= timeout_cycles) begin
                    `uvm_error("RD_TIMEOUT",
                        $sformatf("RVALID timeout! addr=0x%0h after %0d cycles",
                                  trans.addr, timeout_cycles))
                    v_if.drive_cb.RREADY <= 1'b0;
                    return;
                end
            end
            trans.data = v_if.drive_cb.RDATA;
            trans.rresp = v_if.drive_cb.RRESP;
            @(v_if.drive_mp.drive_cb);
            v_if.drive_cb.RREADY <= 1'b0;
```

```
        end
    endtask

    task write_item(axi_lite_transaction trans); endtask
endclass
```

【模版套用说明】

AR 握手后等 RVALID，含超时计数器和 reset 豁免。RREADY 在 ARREADY 握手完成后拉高（表示 master 已准备好接收读数据）。模版 process_trans + monitor_reset 结构完整保留，仅 drive_item 转发到 read_item。

题目 19 AXI-Lite 完整 Master Driver（读写并行）

难度：高级

标签：fork/join、通道并行、队列关联、死锁规避、响应回传

【题目描述】

在单个 driver 内实现读写通道真正并行，避免相互阻塞。

架构设计：

1. fetch_items 线程：负责从 sequencer 取 item，按 is_write 分发到 wr_q / rd_q 两个队列，立即 item_done；

2. wr_channel 线程：监听 wr_q，取出写事务 -> fork AW/W 并行 -> join -> B 通道；

3. rd_channel 线程：监听 rd_q，取出读事务 -> AR 通道 -> R 通道；

4. monitor_reset 线程：清所有 VALID/READY 信号；

5. 死锁规避：读写通道完全独立，B/R 通道各自等待各自的 BVALID/RVALID，互不阻塞；

6. 两个队列用 semaphore 保护，防止 fetch 线程和 channel 线程同时操作。

【完整答案代码】

```
class my_driver extends uvm_driver#(axi_lite_transaction);
    virtual axi_lite_interface v_if;
    axi_lite_transaction wr_q[$], rd_q[$];
    semaphore wr_sem, rd_sem;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        wr_sem = new(1); rd_sem = new(1);
        if (!uvm_config_db#(virtual axi_lite_interface)::get(
                this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "AXI-Lite interface not found")
    endfunction

    // 四线程并发
    task run_phase(uvm_phase phase);
        fork
            fetch_items();
            wr_channel();
            rd_channel();
            monitor_reset();
        join
    endtask

    task monitor_reset();
```

```
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.AWVALID <= 0; v_if.drive_cb.WVALID <= 0;
            v_if.drive_cb.BREADY <= 0; v_if.drive_cb.ARVALID <= 0;
            v_if.drive_cb.RREADY <= 0;
        end
    endtask

    task fetch_items();
        axi_lite_transaction req;
        forever begin
            seq_item_port.get_next_item(req);
            if (req.is_write) begin
                wr_sem.get(); wr_q.push_back(req); wr_sem.put();
            end else begin
                rd_sem.get(); rd_q.push_back(req); rd_sem.put();
            end
            seq_item_port.item_done();
        end
    endtask

    task wr_channel();
        axi_lite_transaction req;
        forever begin
            wait(wr_q.size() > 0);
            wr_sem.get(); req = wr_q.pop_front(); wr_sem.put();
            fork send_aw(req); send_w(req); join
            recv_b(req);
        end
    endtask

    task rd_channel();
        axi_lite_transaction req;
        forever begin
            wait(rd_q.size() > 0);
            rd_sem.get(); req = rd_q.pop_front(); rd_sem.put();
            send_ar(req); recv_r(req);
        end
    endtask

    task send_aw(axi_lite_transaction t);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.AWVALID <= 1; v_if.drive_cb.AWADDR <= t.addr;
        do @(v_if.drive_mp.drive_cb); while (!v_if.drive_cb.AWREADY);
        v_if.drive_cb.AWVALID <= 0;
    endtask

```

```
    task send_w(axi_lite_transaction t);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.WVALID <= 1; v_if.drive_cb.WDATA <= t.data;
        v_if.drive_cb.WSTRB <= t.wstrb;
        do @(v_if.drive_mp.drive_cb); while (!v_if.drive_cb.WREADY);
        v_if.drive_cb.WVALID <= 0;
    endtask

    task recv_b(axi_lite_transaction t);
        v_if.drive_cb.BREADY <= 1;
        do @(v_if.drive_mp.drive_cb); while (!v_if.drive_cb.BVALID);
        t.bresp = v_if.drive_cb.BRESP;
        @(v_if.drive_mp.drive_cb); v_if.drive_cb.BREADY <= 0;
    endtask

    task send_ar(axi_lite_transaction t);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.ARVALID <= 1; v_if.drive_cb.ARADDR <= t.addr;
        do @(v_if.drive_mp.drive_cb); while (!v_if.drive_cb.ARREADY);
        v_if.drive_cb.ARVALID <= 0;
    endtask

    task recv_r(axi_lite_transaction t);
        v_if.drive_cb.RREADY <= 1;
        do @(v_if.drive_mp.drive_cb); while (!v_if.drive_cb.RVALID);
        t.data = v_if.drive_cb.RDATA; t.rresp = v_if.drive_cb.RRESP;
        @(v_if.drive_mp.drive_cb); v_if.drive_cb.RREADY <= 0;
    endtask

    task drive_item(axi_lite_transaction t); endtask
    task read_item(axi_lite_transaction t); endtask
    task write_item(axi_lite_transaction t); endtask
endclass
```

【模版套用说明】

run_phase fork 四线程：fetch（取 item 分发）+ wr_channel + rd_channel + monitor_reset。读写通道独立运行，用 semaphore
保护共享队列。无死锁：两个通道各自独立等待自己的队列项，B/R 通道独立互不阻塞。

题目 20 Burst Driver：Beat 计数与地址递增

难度：高级

标签：burst、beat 计数、地址递增、strobe、backpressure

【题目描述】

实现将单笔 burst 事务拆分为多个 beat 连续驱动的 burst driver。

地址递增规则（INCR 模式）：beat_n 的地址 = base_addr + n * beat_bytes。

关键实现要点：

1. beat 循环：for(beat=0; beat<burst_len; beat++) 驱动每个 beat；

2. 地址递增：每个 beat 的 addr = base_addr + beat * beat_bytes；

3. last 信号：仅在最后一拍（beat == burst_len-1）拉高，其余拍保持为 0；

4. strobe 数组：直接使用事务中的 strobe[beat]，若最后拍不足整拍，strobe 对应位为 0；

5. backpressure：每个 beat 发出后 do-while 等待 ready，期间保持所有信号稳定；

6. 可选随机 idle：valid 拉低若干拍后再发下一个 beat，用于压力测试。

【完整答案代码】

```
class my_driver extends uvm_driver#(burst_transaction);
    virtual my_interface v_if;
    bit idle_insert_en = 0;
    int max_idle = 3;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "interface not found")
        void'(uvm_config_db#(bit)::get(this, "", "idle_insert_en", idle_insert_en));
        void'(uvm_config_db#(int)::get(this, "", "max_idle", max_idle));
    endfunction

    task run_phase(uvm_phase phase);
        fork process_trans(); monitor_reset(); join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.valid <= 0;
            v_if.drive_cb.data <= '0;
            v_if.drive_cb.last <= 0;
        end
```

```
    endtask

    task process_trans();
        burst_transaction req, rsp;
        forever begin
            seq_item_port.get_next_item(req);
            drive_item(req);
            void'($cast(rsp, req.clone()));
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.item_done(rsp);
        end
    endtask

    task drive_item(burst_transaction trans);
        for (int beat = 0; beat < trans.burst_len; beat++) begin
            bit is_last = (beat == trans.burst_len - 1);
            bit [31:0] beat_addr = trans.base_addr + beat * trans.beat_bytes;

            // 可选：在 beat 之间插入 idle（valid=0）
            if (idle_insert_en && beat > 0) begin
                int idle_cnt = $urandom_range(0, max_idle);
                if (idle_cnt > 0) begin
                    v_if.drive_cb.valid <= 0;
                    repeat(idle_cnt) @(v_if.drive_mp.drive_cb);
                end
            end

            write_item_beat(beat_addr, trans.data[beat],
                            trans.strobe[beat], is_last);
        end
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.valid <= 0;
        v_if.drive_cb.last <= 0;
    endtask

    task write_item_beat(
        input bit [31:0] addr,
        input bit [31:0] data,
        input bit [3:0] strobe,
        input bit last
    );
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.valid <= 1'b1;
        v_if.drive_cb.addr <= addr;
        v_if.drive_cb.data <= data;
        v_if.drive_cb.strobe <= strobe;
        v_if.drive_cb.last <= last;
```

```
        do @(v_if.drive_mp.drive_cb);
        while (!v_if.drive_cb.ready);
    endtask

    task read_item(burst_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.data[0] = v_if.drive_cb.data;
    endtask
    task write_item(burst_transaction trans); endtask
endclass
```

【模版套用说明】

drive_item 遍历所有 beat，地址 = base_addr + beat*beat_bytes，write_item_beat 等 ready。模版 process_trans + monitor_reset 骨架保留，write_item 扩展为 write_item_beat（带 addr/strobe/last 参数）。

题目 21 Streaming ready/valid Driver（插 idle）

难度：中级

标签：ready/valid、随机 idle、backpressure、稳定保持、数据完整性

【题目描述】

实现 AXI-Stream 风格的 streaming driver，支持随机 idle 插入。

关键协议规则：

1. valid=1 时，payload 和 last 必须保持稳定直到 ready=1（握手完成），期间不能更新数据；

2. 发送完一个 word 后，可选插入若干个 idle 周期（valid=0）再发下一个；

3. 最后一个 word（idx == words.size()-1）拉高 last=1，其余 last=0；

4. 发送完所有 word 后，valid=0，last=0，接口回 idle；

5. 配置参数 idle_insert_en（开关）和 max_idle（最大 idle 拍数）通过 config_db 注入。

【完整答案代码】

```
class my_driver extends uvm_driver#(stream_transaction);
    virtual my_interface v_if;
    bit idle_insert_en = 1;
    int max_idle = 4;
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "interface not found")
        void'(uvm_config_db#(bit)::get(this, "", "idle_insert_en", idle_insert_en));
        void'(uvm_config_db#(int)::get(this, "", "max_idle", max_idle));
    endfunction

    task run_phase(uvm_phase phase);
        fork process_trans(); monitor_reset(); join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.valid <= 0;
            v_if.drive_cb.payload <= '0;
            v_if.drive_cb.last <= 0;
        end
    endtask

    task process_trans();
```

```
        stream_transaction req, rsp;
        forever begin
            seq_item_port.get_next_item(req);
            drive_item(req);
            void'($cast(rsp, req.clone()));
            rsp.set_sequence_id(req.get_sequence_id());
            seq_item_port.item_done(rsp);
        end
    endtask

    task drive_item(stream_transaction trans);
        int n_words = trans.words.size();
        for (int i = 0; i < n_words; i++) begin
            if (idle_insert_en && i > 0) begin
                int idle_cnt = $urandom_range(0, max_idle);
                if (idle_cnt > 0) begin
                    v_if.drive_cb.valid <= 0;
                    repeat(idle_cnt) @(v_if.drive_mp.drive_cb);
                end
            end
            write_item_word(trans.words[i], (i == n_words - 1));
        end
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.valid <= 0;
        v_if.drive_cb.last <= 0;
    endtask

    task write_item_word(
        input bit [31:0] word,
        input bit last
    );
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.valid <= 1'b1;
        v_if.drive_cb.payload <= word;
        v_if.drive_cb.last <= last;
        do @(v_if.drive_mp.drive_cb);
        while (!v_if.drive_cb.ready);
    endtask

    task read_item(stream_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.words[0] = v_if.drive_cb.payload;
    endtask
    task write_item(stream_transaction trans); endtask
endclass
```

【模版套用说明】

write_item_word 发出后 do-while 等 ready，期间 valid/payload/last 完全稳定。drive_item 循环遍历 words，两个 word 之间根据配置插入
idle（valid=0）。模版骨架完整保留，write_item 扩展为 write_item_word（带 last 参数）。

题目 22 可编程 Reset 序列发生器

难度：中级

标签：clk 生成、动态 reset、可复现、事件通知、时序安全

【题目描述】

实现时钟和复位发生器 UVM 组件，为整个 testbench 提供受控的时钟和 reset 信号。

功能要求：

1. 时钟生成：gen_clk task 产生可配置周期（clk_period_ns）和占空比（duty_cycle）的时钟；

2. 初始 reset：仿真启动时 reset_n=0，持续 init_reset_cyc 个时钟周期后释放；

3. do_reset(width_cycles) API：供 test/sequence 调用，注入任意宽度的 reset 脉冲；

4. reset_event 广播：每次注入 reset 时触发 uvm_event，通知其他组件（如 driver 的 monitor_reset 线程）；

5. 时序安全：reset 切换在时钟边沿后进行（@posedge clk -> 再切换 reset_n），防止毛刺；

6. 此组件无 seq_item_port，不适用 pull driver 模版，但接口信号驱动风格一致。

【完整答案代码】

```
class clk_rst_gen extends uvm_component;
    virtual clk_rst_interface v_if;
    uvm_event reset_event;

    real clk_period_ns = 10.0;
    real duty_cycle = 0.5;
    int init_reset_cyc = 10;
    `uvm_component_utils(clk_rst_gen)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        reset_event = new("reset_event");
        if (!uvm_config_db#(virtual clk_rst_interface)::get(
                this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "clk_rst interface not found")
        void'(uvm_config_db#(real)::get(this, "", "clk_period_ns", clk_period_ns));
        void'(uvm_config_db#(int)::get(this, "", "init_reset_cyc", init_reset_cyc));
    endfunction

    task run_phase(uvm_phase phase);
        phase.raise_objection(this);
        fork
            gen_clk();
            init_reset_seq();
        join_none
```

```
        phase.drop_objection(this);
    endtask

    task gen_clk();
        v_if.clk = 0;
        forever begin
            #(clk_period_ns * duty_cycle * 1ns);
            v_if.clk = 1;
            #(clk_period_ns * (1.0 - duty_cycle) * 1ns);
            v_if.clk = 0;
        end
    endtask

    task init_reset_seq();
        v_if.reset_n = 0;
        `uvm_info("CLK_RST", "Initial reset asserted", UVM_LOW)
        repeat(init_reset_cyc) @(posedge v_if.clk);
        @(posedge v_if.clk);
        v_if.reset_n = 1;
        `uvm_info("CLK_RST", "Initial reset released", UVM_LOW)
    endtask

    // API：外部调用注入 reset（时序安全：边沿后切换）
    task do_reset(int width_cycles = 5);
        `uvm_info("CLK_RST",
            $sformatf("Injecting reset for %0d cycles", width_cycles), UVM_LOW)
        reset_event.trigger();
        @(posedge v_if.clk);
        v_if.reset_n = 0;
        repeat(width_cycles) @(posedge v_if.clk);
        @(posedge v_if.clk);
        v_if.reset_n = 1;
        `uvm_info("CLK_RST", "Reset injection complete", UVM_LOW)
    endtask
endclass
```

【模版套用说明】

此组件不适用 pull driver 模版（无 seq_item_port），但接口驱动风格与模版一致。do_reset API 供 test/sequence 调用，reset_event 广播通知其他 driver 的 monitor_reset 线程。时序安全：reset 切换在 @posedge clk 后执行，防止毛刺。

题目 23 Pipeline Driver：提前释放 Sequence + end_event

难度：高级

标签：pipeline、end_event、begin_tr/end_tr、clone、并行相位

【题目描述】

实现真正的 pipeline driver，使地址相位和数据相位可以重叠，提升总线利用率。

Pipeline 实现要点：

1. seq_item_port.get(req)：立即解除 sequence finish_item 阻塞，无需等数据相位完成；

2. 立即 clone（req_snap = req.clone()）：防止 sequence 下一次 randomize 污染 driver 正在使用的数据；

3. 每笔 req 在 fork join_none 中独立执行，支持多笔 req 同时在途；

4. addr_sem 信号量：保证地址相位串行（一次只有一笔 req 占用地址相位资源）；

5. begin_tr/end_tr 标记生命周期：地址相位启动时 begin_tr，数据相位完成时 end_tr（触发 end_event）；

6. 响应通过 put_response 异步回传，与 get() 配对（不用 item_done）。

【完整答案代码】

```
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    semaphore addr_sem; // 保证地址相位互斥
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        addr_sem = new(1);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "interface not found")
    endfunction

    task run_phase(uvm_phase phase);
        fork process_trans(); monitor_reset(); join
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.data <= 8'd0;
        end
    endtask

    // process_trans：get() 提前释放 sequence，每笔 req fork 独立执行
```

```
    task process_trans();
        my_transaction req, req_snap, rsp;
        forever begin
            seq_item_port.get(req);

            // 立即 clone，防止 sequence 重随机化污染数据
            $cast(req_snap, req.clone());
            accept_tr(req_snap, $realtime);

            // fork join_none：每笔 req 在独立线程中执行
            fork
                automatic my_transaction r = req_snap;
                automatic my_transaction r_ori = req;
                begin
                    // 地址相位：互斥
                    addr_sem.get();
                    void'(begin_tr(r, "pipeline_drv"));
                    @(v_if.drive_mp.drive_cb);
                    v_if.drive_cb.data <= r.addr; // 发地址（示意）
                    @(v_if.drive_mp.drive_cb);
                    addr_sem.put(); // 释放：允许下一笔进入地址相位

                    // 数据相位（与下一笔地址相位重叠）
                    drive_item(r);

                    // 事务完成：触发 end_event
                    end_tr(r, $realtime);

                    $cast(rsp, r.clone());
                    rsp.set_sequence_id(r_ori.get_sequence_id());
                    seq_item_port.put_response(rsp);
                end
            join_none
        end
    endtask

    task drive_item(my_transaction trans);
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
    endtask

    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.data = v_if.drive_cb.data;
    endtask
```

```
    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
    endtask
endclass
```

【模版套用说明】

get() + clone() 是 pipeline driver 两个关键点。fork join_none 让每笔 req 在独立线程执行，addr_sem 保证地址相位串行。end_tr 触发 end_event，sequence 可 wait_on() 等待。sequence_id 从原始 req（非 clone）获取。

题目 24 Multi-outstanding：ID 路由与响应回传

难度：高级

标签：multi-outstanding、ID routing、节流、关联表、响应回传

【题目描述】

实现支持多笔在途（multi-outstanding）的 driver，通过 ID 路由将响应关联回对应请求。

架构设计：

1. inflight[id] 关联表：记录每个 id 对应的 req 上下文，响应到来时按 id 查找；

2. slot_sem 信号量：初始值 = max_outstanding，发一笔请求 get() 一个槽位，收到响应 put() 归还，实现节流；

3. 两线程分离：process_trans 取 item 发请求登记，recv_responses 监听接口响应按 id 回传；

4. inflight_sem 保护关联表，防止两线程并发访问竞争；

5. 响应到来时从 inflight 删除条目（防止内存泄漏），并归还 slot_sem 槽位（解除节流）；

6. 乱序响应完全支持，因为 id 唯一标识每笔事务。

【完整答案代码】

```
class my_driver extends uvm_driver#(my_transaction);
    virtual my_interface v_if;
    int max_outstanding = 4;

    my_transaction inflight[int]; // in-flight 关联表：id -> req
    semaphore inflight_sem;
    semaphore slot_sem; // 节流信号量
    `uvm_component_utils(my_driver)

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual my_interface)::get(this, "", "vif", v_if))
            `uvm_fatal("NO_VIF", "interface not found")
        void'(uvm_config_db#(int)::get(this, "", "max_outstanding",
max_outstanding));
        inflight_sem = new(1);
        slot_sem = new(max_outstanding);
    endfunction

    task run_phase(uvm_phase phase);
        fork
            process_trans();
            recv_responses();
            monitor_reset();
        join
```

```
    endtask

    task monitor_reset();
        forever begin
            @(negedge v_if.mon_cb.reset);
            v_if.drive_cb.data <= 8'd0;
        end
    endtask

    task process_trans();
        my_transaction req;
        forever begin
            slot_sem.get(); // 节流：超过 max_outstanding 时阻塞
            seq_item_port.get_next_item(req);
            inflight_sem.get();
            inflight[req.id] = req;
            inflight_sem.put();
            drive_item(req);
            seq_item_port.item_done();
        end
    endtask

    task recv_responses();
        my_transaction rsp;
        forever begin
            do @(v_if.drive_mp.drive_cb);
            while (!v_if.drive_cb.rsp_valid);
            begin
                int rsp_id = v_if.drive_cb.rsp_id;
                inflight_sem.get();
                if (!inflight.exists(rsp_id)) begin
                    `uvm_error("DRV", $sformatf("Unexpected rsp id=%0d", rsp_id))
                    inflight_sem.put();
                end else begin
                    $cast(rsp, inflight[rsp_id].clone());
                    rsp.set_sequence_id(inflight[rsp_id].get_sequence_id());
                    rsp.data = v_if.drive_cb.rsp_data;
                    inflight.delete(rsp_id);
                    inflight_sem.put();
                    seq_item_port.put_response(rsp);
                    slot_sem.put(); // 归还槽位，解除节流
                end
            end
        end
    endtask

    task drive_item(my_transaction trans);
```

```
        case(trans.type)
            0: read_item(trans);
            1: write_item(trans);
        endcase
    endtask

    task read_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        trans.data = v_if.drive_cb.data;
    endtask

    task write_item(my_transaction trans);
        @(v_if.drive_mp.drive_cb);
        v_if.drive_cb.data <= trans.data;
    endtask
endclass
```

【模版套用说明】

slot_sem 实现节流（max_outstanding 个槽位），inflight 关联表用 semaphore 保护。req 侧 item_done() 不带 rsp（rsp 由响应线程异步 put_response 回传）。响应处理后 inflight.delete() 防泄漏，slot_sem.put() 解除节流。
